Fast Bounds on Matching Decoder Soft Output for the Surface Code

Sumeet Shirgure and Siyuan Niu

Department of Electrical and Computer Engineering, University of Central Florida, Orlando, FL 32816, USA

Surface code decoders like Minimum Weight Perfect Matching (MWPM) can also be used to estimate or bound the probability of a logical failure conditioned on the error syndrome commonly called "soft outputs". Estimates like the complementary gap of a matching, or others like the "cluster gap" are used to produce high fidelity magic states via post-selection. They are also helpful in reducing the overhead of fault-tolerant computation via code concatenation schemes like yoked surface codes. In this work we provide techniques to tightly lower bound the complementary gap of a rotated surface code syndrome block and provide fast parallel algorithms for computing this lower bound efficiently. Our algorithms run in real time for the moderate distances and reasonable physical noise rates required by these applications. We achieve latencies of the order of one to tens of microseconds on sufficiently fast CPU cores.

1 Introduction

A decoder for a quantum error-correcting code maps a measured syndrome to a correction. Many applications need more than the correction: they need to know how much to *trust* it. Such per-shot reliability information, usually called *soft output*, was used for the decoding of concatenated codes already in Ref. [18], where each inner block passes the likelihood of its logical outcomes, rather than a hard decision, to the outer decoder.

For the surface code [2, 6] decoded by minimum-weight perfect matching (MWPM), the most widely used soft output is the *complementary gap* [1, 10, 14]. Let $W^\star$ be the weight of the minimum-weight correction and W^c the weight of the minimum-weight correction in the *other* logical class. Because edge weights are negative log-likelihoods, the gap $\Delta = W^c - W^\star \geq 0$ is the log-likelihood ratio of the two most likely explanations of the syndrome that differ by a logical operator. A small gap signals an ambiguous syndrome, and the conditional probability of a logical failure falls roughly exponentially in Δ.

The complementary gap has become a central ingredient of several low-overhead fault-tolerance schemes.

- **Post-selection for magic states.** Fault-tolerant post-selection on the gap was proposed for magic state preparation in Ref. [1]. Magic state cultivation [9] grows a T state inside a colour code, escapes it into a large matchable code and discards the attempt if the gap of the escape-stage matching is below a threshold. Refinements of cultivation [13] rely on the same post-selection step.

Sumeet Shirgure: sumeetparameshwar.shirgure@ucf.edu

Siyuan Niu: siyuan.niu@ucf.edu

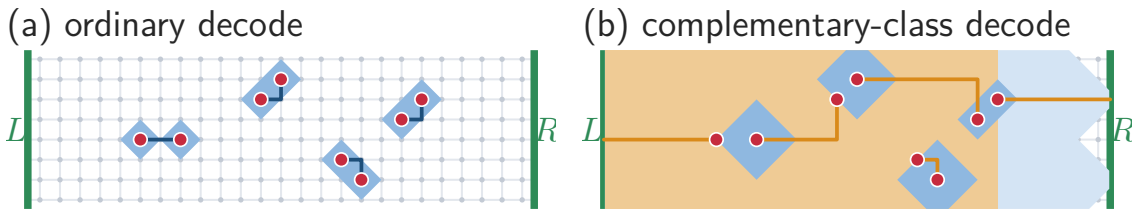


Figure 1: The region that each decode has to grow, on a 2D slice of a detector graph with unit edge weights whose left and right sides are the two logical boundaries L and R. The shot has eight defects (dots) in four tight pairs, far from both boundaries. (a) The ordinary decode grows a region of radius ρ_v (the dual variables of Sec. 2.3) around each defect and stops as soon as every defect is matched (black lines); it visits only the shaded nodes. (b) The complementary decode must also produce a matching in the other logical class. Its extra detector is attached to the whole L boundary, so its region grows from that entire side (orange) across the empty space between the clusters until the forced logical chain closes. Here $\Delta = 20$, and the complementary decode reaches 198 of the 200 nodes, against 33 for the ordinary decode. Both panels show the dual radii read from PyMatching at the end of each decode.

- **Code concatenation.** Yoked surface codes [10] concatenate surface-code patches with simple outer parity codes and use each patch's gap as the edge weight of an outer matching problem, roughly doubling (1D) or quadrupling (2D) the effective code distance. The hierarchical memories of Pattison, Krishna and Preskill [17] concatenate a constant-rate quantum LDPC outer code with surface-code inner patches. The long-range checks of the outer code are then carried out by moving surface-code-encoded qubits with nearest-neighbour gates in two dimensions, and the logical failure rate is suppressed superpolynomially in the distance below threshold. Their analysis uses only hard decisions from the inner decoder. Meister, Pattison and Preskill [16] showed that feeding the inner soft output to the outer decoder as a prior instead raises the pseudothreshold of such constructions substantially.

The gap is usually computed by decoding *twice*: once without constraint, and once with the logical class forced, for instance by turning the logical observable into an extra detector that is switched on or off [9, 10]. The second decode is expensive. To force the other class, the matching decoder must grow a region from the entire boundary sheet of the patch across the spacetime volume until the accumulated dual reaches Δ. On the high-gap shots an application keeps, almost all of that growth happens in the empty space between the small, well-separated clusters of the syndrome, which the first decode never had to visit (Fig. 1). In our benchmarks the complementary decode is between 7× and 67× slower than the ordinary decode it annotates (Sec. 7).

Several lines of work address this cost. Meister, Pattison and Preskill [16] introduced soft outputs derived from the clusters of a union-find-style decoder (the "cluster gap"), which avoid a second decode. Computing the cluster gap, however, still requires growing clusters through the empty space between the syndrome's clusters until they merge with each other or reach a boundary. It therefore inherits much of the volume sweep of the second decode, and because this growth is a sequence of dependent merge events, it is hard to parallelise on a CPU. Kishi *et al.* [15] improved their efficiency further with extra-cluster growth and early stopping, but real-time implementations of these approaches require extensive resources like high-end FPGAs with a processing unit attached to every node in the detector graph.

This work. We show that a *certified lower bound* on the complementary gap can be read off the final state of a single, ordinary MWPM decode, without ever sweeping the

empty volume. Our contributions are:

1. **A reduced-cost characterisation of the gap** (Sec. 3). The edges in which the best matching and the best matching of the other logical class differ form a single path between the two logical boundaries, alternating between edges of the two matchings. The blossom algorithm already maintains a dual solution of the matching linear program (one number per defect and one per blossom). Using it, we rewrite the cost of any such path as a sum of *non-negative reduced costs*, plus a non-negative term that only blossoms contribute. This turns the gap into a shortest-path problem with non-negative edge lengths on a complete graph whose nodes are the defects and the two boundaries. A single dense Dijkstra search solves it, and its value never exceeds the gap.

2. **Volume-free pricing** (Sec. 4). The reduced cost of a pair of defects needs their graph distance, which the decode never computes for far-apart pairs. We replace it by a lower bound derived from distances to a few landmark vertices [11], precomputed once per detector error model. The search then still returns a rigorous lower bound on the gap, now without visiting the empty spacetime volume at all.

3. **Distance preservation** (Sec. 5). We prove that on every shot, blossoms included, the bound is at least the weight of the lightest logical operator minus twice the weight of the minimum-weight correction. The exact gap satisfies the same inequality, and it is the one that a minimum-weight outer decoder needs on patches that were decoded correctly. Using the bound in place of the gap therefore keeps the doubled (1D) and quadrupled (2D) distance exponents of yoked surface codes.

4. **A fast parallel implementation** (Sec. 6), *fastgap*, built inside a fork of PyMatching [12]. It reproduces PyMatching's predictions and weights, and computes the gap bound in 0.1–10 μs (median, one thread) on rotated surface-code memory experiments with $d \leq 17$ at $p \in \{10^{-3}, 5 \times 10^{-3}\}$, 11–51$\times$ faster than the complementary decode of the standard two-decode method (Sec. 7).

2 Background

2.1 Detector graphs and matching

We follow the detector-error-model formalism of Stim [8]. A circuit with its noise model defines a set V of detectors and a list of independent error mechanisms. After decomposition, every mechanism flips at most two detectors, and it may also flip the logical observable. The *detector graph* G has a node per detector, a virtual boundary node b, and an edge per error mechanism e (with one endpoint b if it flips a single detector) of weight $w_e = \ln \frac{1-p_e}{p_e}$. In a shot, the set $D \subseteq V$ of detectors that fire are the *defects*. A correction is a set of edges whose boundary is D, and the most likely one (under the independent-edge approximation) has minimum total weight. Equivalently, it is a minimum-weight perfect matching of the defects on the complete graph in which defects u, v are joined at cost $\mathrm{dist}(u, v)$, the shortest-path distance in G, and any defect can be matched to the boundary [2, 6]. We use PyMatching v2 and its sparse blossom algorithm [12], which solves this problem without building the complete graph, by growing regions on G itself. Internally, PyMatching discretises weights to even integers; all quantities in this paper are in those integer units, and the bounds we prove are bounds on the gap of the discretised problem that PyMatching actually solves.

2.2 The complementary gap

Let $M^\star$ be a minimum-weight matching, of weight $W^\star$, and M^c a minimum-weight matching whose correction lies in the other logical class, of weight W^c.

Definition 1. The *complementary gap* of a shot is $\Delta = W^c - W^\star \geq 0$.

Since weights are negative log-likelihoods, Δ is the log-likelihood ratio between the two most likely corrections in different logical classes, and the probability that the decoder's choice is wrong is commonly approximated by $1/(1+e^{\Delta})$. Applications usually report it in decibels, $\Delta_{\mathrm{dB}} = 10\log_{10}(e)\,\Delta$, and we do the same in the figures.

2.3 The matching linear program

Consider the complete graph whose nodes are the defects D, with an edge (u, v) of length $\mathrm{dist}(u, v)$ for every pair of defects and a boundary edge (u, b) of length $\mathrm{dist}(u, b)$ for every defect. We say that an edge *crosses* a set $S \subseteq D$ if exactly one of its endpoints lies in S; a boundary edge (u, b) crosses every set that contains u. Write $\delta(S)$ for the set of edges that cross S, and $\delta(v) = \delta(\{v\})$. With $\mathcal{O} = \{S \subseteq D : |S| \geq 3 \text{ odd}\}$, the matching linear program and its dual are [4]

$$
\begin{aligned}
\text{(P)} \qquad \min \ & \sum_{e} \mathrm{dist}(e)\, x_e \\
\text{s.t.} \ & \sum_{e \in \delta(v)} x_e = 1 && (v \in D), \\
& \sum_{e \in \delta(S)} x_e \geq 1 && (S \in \mathcal{O}), \\
& x_e \geq 0, \\
\text{(D)} \qquad \max \ & \sum_{v \in D} y_v + \sum_{S \in \mathcal{O}} y_S \\
\text{s.t.} \ & y_u + y_v + \sum_{S \in \mathcal{O}:\, (u,v) \in \delta(S)} y_S \leq \mathrm{dist}(u, v) && (u, v \in D), \\
& y_u + \sum_{S \in \mathcal{O}:\, S \ni u} y_S \leq \mathrm{dist}(u, b) && (u \in D), \\
& y_S \geq 0 && (S \in \mathcal{O}). \qquad (1)
\end{aligned}
$$

The boundary node has no degree constraint: it may absorb any number of defects. Every perfect matching of the defects, with any number of them matched to the boundary, is a 0/1 point of (P), and Edmonds showed that the odd-set constraints make the optimum of (P) integral, so its value is $W^\star$.

Edmonds' blossom algorithm [4, 5] and its sparse variant maintain a feasible solution of (D). An odd set S with $y_S > 0$ is called a *blossom*, and the family of blossoms is laminar: two blossoms are nested or disjoint. It can therefore be drawn as a forest whose leaves are the defects and whose internal nodes are the blossoms, each blossom being the parent of its maximal sub-blossoms and of the defects that lie in no sub-blossom (Fig. 3). Define, for defects u, v,

$$
\rho_u = y_u + \sum_{S \ni u} y_S, \qquad\qquad \beta(u, v) = \sum_{S \ni u,\, S \ni v} y_S, \qquad (2)
$$

the *total radius* of u and the *shared mass* of u and v. The left-hand sides of the constraints of (D) are the total dual *charged* to an edge: $\kappa(u, v) = \rho_u + \rho_v - 2\beta(u, v)$ for a defect pair,

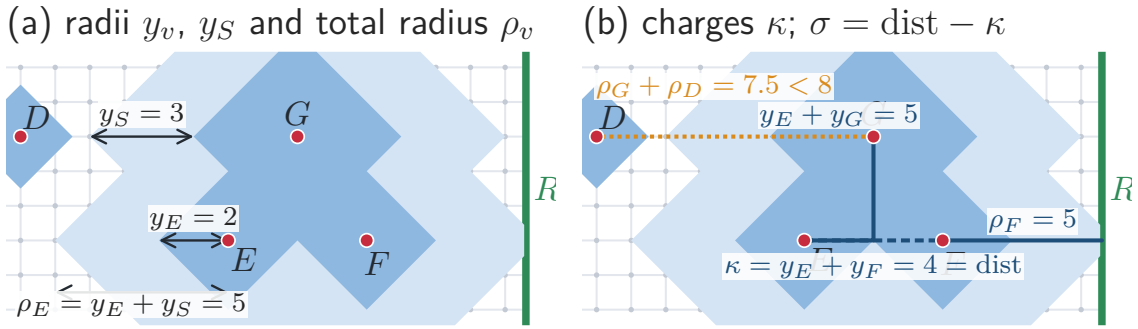


Figure 2: Dual variables of the matching LP, read from PyMatching at the end of the decode of Fig. 4 (right part of the lattice, unit edge weights). (a) The defects E, F, G form a blossom S. Each grows a ball of radius y_v (dark), and the blossom adds a shell of thickness $y_S = 3$ (light) around the union of their balls. The total radius $\rho_E = y_E + y_S$ is how far the region of E reaches. (b) An edge between two defects of the same blossom does not cross S, so it is charged only $\kappa(E, F) = y_E + y_F = \rho_E + \rho_F - 2\beta(E, F)$, with shared mass $\beta(E, F) = y_S$. An edge that leaves S is charged the full total radius of its endpoints: ρ_F for the boundary edge of F (the base of S), and $\kappa(G, D) = \rho_G + \rho_D = 7.5$ for the edge to the defect D outside S, whose length is 8. The reduced cost σ of an edge is its length minus its charge: it is 0 on the matched edges (solid) and on the blossom's cycle edge E–F (dashed), and $\sigma(G, D) = 0.5$.

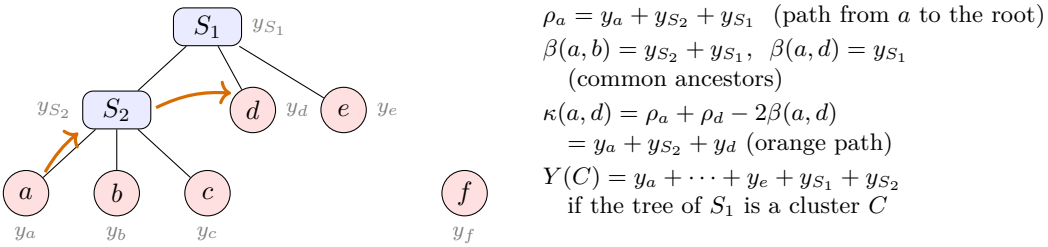


Figure 3: The laminar family of blossoms drawn as a forest: leaves are defects, internal nodes are blossoms, and every node carries its dual value. Here the blossom $S_2 = \{a, b, c\}$ is nested inside $S_1 = \{a, b, c, d, e\}$, and f lies in no blossom. All the dual quantities used in this paper are sums of node values over parts of this forest (right).

because the blossoms that contain both u and v are not crossed, and ρ_u for a boundary edge. Figure 2 shows these quantities geometrically: in sparse blossom each defect grows a ball of radius y_v, and each blossom grows a shell of thickness y_S around the union of its members' regions, so ρ_u is how far the region of u reaches from u. In the forest of Fig. 3, ρ_u is the sum of the values on the path from leaf u to its root, $\beta(u, v)$ is the sum over the common ancestors of u and v, and the charge $\kappa(u, v)$ is the sum over the nodes on the tree path from u to v strictly below their lowest common ancestor. The *reduced costs* are the slacks of the constraints of (D),

$$\sigma(u, v) = \mathrm{dist}(u, v) - \rho_u - \rho_v + 2\beta(u, v), \qquad \sigma(u, b) = \mathrm{dist}(u, b) - \rho_u. \tag{3}$$

When sparse blossom terminates [12], the following standard LP facts hold:

- *Feasibility:* $\sigma \geq 0$ for every defect pair and every boundary edge.
- *Tightness:* $\sigma = 0$ on every edge of $M^\star$.
- *Strong duality:* $W^\star = \sum_v y_v + \sum_S y_S$.

- *Complementary slackness:* if $y_S > 0$, exactly one edge of $M^\star$ crosses S. We call it the *base* of S.

In sparse blossom, y_v is the radius of the region grown from defect v and y_S is the extra radius of blossom region S. These are available at the end of the decode at no extra cost.

3 The gap as a non-negative shortest path

3.1 The two logical boundaries

In a rotated surface-code memory experiment, the detectors of one basis form a detector graph whose boundary edges lie on two opposite sides of the patch, and a logical error is a chain of edges that connects the two sides. We call the two sides L and R, and choose the logical observable to be a representative that runs along the L side, so that it is flipped exactly by the boundary edges on L.[1] The boundary node b thus splits into two nodes, L and R, and the logical class of a matching is the parity of the number of its edges that end on L.

Write $\mathrm{dist}(u, L)$ and $\mathrm{dist}(u, R)$ for the length of a shortest path from defect u to the L side and to the R side, and $d_{LR} = \min_x \left(\mathrm{dist}(x, L) + \mathrm{dist}(x, R)\right)$ for the length of a shortest path from L to R, which is the weight of the lightest logical operator. We allow matchings to contain a bare L–R edge of cost d_{LR}, which represents a logical operator applied with no defect involved. $M^\star$ never uses one, and M^c uses at most one. Since $\mathrm{dist}(u, b) = \min(\mathrm{dist}(u, L), \mathrm{dist}(u, R))$, the boundary reduced costs $\sigma(u, X) = \mathrm{dist}(u, X) - \rho_u$ for $X \in \{L, R\}$ are non-negative by feasibility, and they vanish on the boundary edges of $M^\star$ by tightness.

3.2 The symmetric difference is one alternating path

Write $\mathrm{mate}(u) \in D \cup \{L, R\}$ for the partner of defect u in $M^\star$. In the symmetric difference $M^\star \oplus M^c$, we call an edge of $M^c \setminus M^\star$ a *hop* and an edge of $M^\star \setminus M^c$ a *relay*. An *alternating L–R path* is a path that starts at L, ends at R, whose interior vertices are distinct defects, and whose edges alternate between hops and relays, the relays being exactly the $M^\star$ edges between consecutive defects. Its end edges may be of either kind: a defect that $M^\star$ matches to L can be released by a relay at the start. The bare edge $L \to R$ (a hop of cost d_{LR}) is also an alternating path. Define

$$\mathrm{cost}(P) = \sum_{\text{hops } h \in P} \mathrm{dist}(h) \; - \sum_{\text{relays } r \in P} \mathrm{dist}(r). \tag{4}$$

Lemma 2. $\Delta = \min_P \mathrm{cost}(P)$, *the minimum over alternating L–R paths.*

Proof. For any alternating path P, $M^\star \oplus P$ is a valid matching with exactly one more or one fewer edge at L, hence in the complementary class, of weight $W^\star + \mathrm{cost}(P)$. So $\Delta \le \min_P \mathrm{cost}(P)$.

Conversely, every defect has one edge in each of $M^\star$ and M^c, so it has degree 0 or 2 in $M^\star \oplus M^c$, while L and R may have any degree. Cutting $M^\star \oplus M^c$ at L and R decomposes it into edge-disjoint, defect-disjoint *segments*: cycles through defects only, and paths whose two ends lie in $\{L, R\}$. Toggling a segment Q in $M^\star$ yields a valid matching of weight

[1] A Stim detector error model may attach the observable to interior edges as well. A relabelling that changes no edge and no weight moves it onto the boundary edges of one side; our implementation applies it automatically.

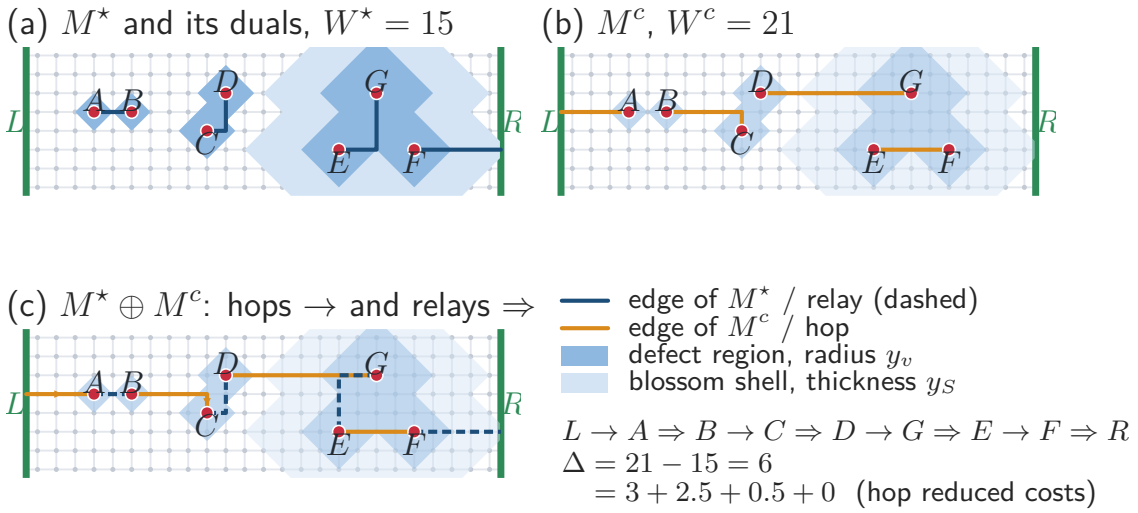


Figure 4: A worked example on a 2D slice with unit edge weights and logical boundaries L (left) and R (right), decoded by PyMatching. The seven defects form three clusters: the pairs $\{A, B\}$ and $\{C, D\}$, and the blossom $S = \{E, F, G\}$ with $y_S = 3$, whose base is the boundary edge of F. (a) The minimum-weight matching $M^\star = \{AB, CD, EG, FR\}$, $W^\star = 2 + 3 + 5 + 5 = 15$, with the dual regions read from the decoder: balls of radius y_v around the defects ($y_A = y_B = 1$, $y_C = y_D = 1.5$, $y_E = y_F = 2$, $y_G = 3$) and the blossom shell of thickness y_S. Strong duality holds: $\sum_v y_v + y_S = 15$. (b) The best matching in the other logical class, $M^c = \{LA, BC, DG, EF\}$, $W^c = 4+5+8+4 = 21$. (c) Their symmetric difference is the single alternating path $L \to A \Rightarrow B \to C \Rightarrow D \to G \Rightarrow E \to F \Rightarrow R$ (hops $\to$, relays $\Rightarrow$), of cost $\Delta = 21 - 15 = 6$. Pricing each hop by its reduced cost gives $\sigma(L, A) + \sigma(B, C) + \sigma(D, G) + \sigma(E, F) = 3 + 2.5 + 0.5 + 0 = 6$ as well. The last hop stays inside the blossom: its reduced cost $4 - \rho_E - \rho_F + 2\beta(E, F) = 4 - 5 - 5 + 6$ is 0 only thanks to the shared mass $\beta(E, F) = y_S$.

$W^\star + \mathrm{cost}(Q)$, and it changes the class iff Q is an L–R path. Since $M^\star \oplus M^c$ changes the class, it contains an odd number of L–R segments $P_1, \ldots, P_{2m+1}$, ordered so that P_1 has the least cost. Every other segment has non-negative cost (toggling it alone stays in the class of $M^\star$), and so has every pair P_{2i}, P_{2i+1} (toggling both stays in the class). Because toggles of disjoint segments add,

$$\Delta = W^c - W^\star = \sum_Q \mathrm{cost}(Q) \;\geq\; \mathrm{cost}(P_1) \;\geq\; \min_P \mathrm{cost}(P). \qquad \square$$

Lemma 2 is a shortest-path problem, but not yet one Dijkstra can solve: relays carry negative weight and the path must alternate. Fig. 4 shows a small example.

3.3 Reduced costs

Theorem 3. *For every alternating L–R path P,*

$$\mathrm{cost}(P) = \sum_{\text{hops } h \in P} \sigma(h) \;+\; 2 \sum_S y_S \, n_S(P) \;\geq\; \sum_{\text{hops } h \in P} \sigma(h), \tag{5}$$

where $n_S(P)$ is the number of maximal runs of P inside the blossom S that are both entered and left by hops. It satisfies $0 \leq n_S(P) \leq (|S| - 1)/2$.

Proof. Write the path as $L = z_0, z_1, \ldots, z_m, z_{m+1} = R$ with edges $e_i = (z_{i-1}, z_i)$, and set $\rho_L = \rho_R = 0$ and $\beta(e) = 0$ for every edge that touches L or R. With this convention the

charge of every edge is $\kappa(e_i) = \rho_{z_{i-1}} + \rho_{z_i} - 2\beta(e_i)$, both for defect pairs and for boundary edges, and the definition (3) of the reduced cost reads

$$\operatorname{dist}(e) = \sigma(e) + \rho_a + \rho_b - 2\beta(e) \qquad \text{for every edge } e = (a, b). \tag{6}$$

Relays are edges of $M^\star$ and hence tight, $\sigma(r) = 0$. Substituting (6) into the definition of $\operatorname{cost}(P)$,

$$\begin{aligned}\operatorname{cost}(P) &= \sum_{\text{hops } h=(a,b)} (\sigma(h) + \rho_a + \rho_b - 2\beta(h)) - \sum_{\text{relays } r=(a,b)} (\rho_a + \rho_b - 2\beta(r)) \\ &= \sum_{\text{hops } h} \sigma(h) + \Big[\sum_{\text{hops } (a,b)} (\rho_a + \rho_b) - \sum_{\text{relays } (a,b)} (\rho_a + \rho_b) \Big] \\ &\quad + 2\Big[\sum_{\text{relays } r} \beta(r) - \sum_{\text{hops } h} \beta(h) \Big].\end{aligned}$$

Every defect z_i on P is an endpoint of exactly two edges of P, one hop and one relay, so ρ_{z_i} appears once in each of the two sums of the first bracket and cancels; the end points contribute $\rho_L = \rho_R = 0$. The first bracket therefore vanishes, and

$$\operatorname{cost}(P) = \sum_{\text{hops}} \sigma(h) + 2\Big[\sum_{\text{relays } r} \beta(r) - \sum_{\text{hops } h} \beta(h) \Big].$$

An edge contributes y_S to β exactly when both its endpoints lie in S, so the bracket equals $\sum_S y_S\,(r_S - h_S)$, with r_S and h_S the numbers of relays and hops of P inside S. Since $L, R \notin S$, the vertices of P in S form maximal runs (*segments*), each entered and left by an edge crossing S. Inside a segment, hops and relays alternate. If both crossing edges are hops, the segment's inner edges begin and end with relays, contributing $+1$ to $r_S - h_S$. A relay that crosses S must be the base of S (complementary slackness), and a simple path uses the base at most once, so at most one crossing edge is a relay; in that case the inner edges contain equally many hops and relays, contributing 0. Hence $r_S - h_S = n_S(P) \geq 0$.

For the upper bound, every segment that counts towards $n_S(P)$ contains at least one relay inside S, and distinct segments contain distinct relays. These relays are edges of $M^\star$ with both endpoints in S. Since S is odd and exactly one edge of $M^\star$ (the base) leaves it, $M^\star$ has exactly $(|S| - 1)/2$ edges inside S, so $n_S(P) \leq (|S| - 1)/2$. □

In particular $n_S(P) \in \{0, 1\}$ for a blossom of three defects, as in Fig. 4. For larger blossoms $n_S(P) \leq 1$ does not hold in general: a simple path can enter a blossom of five defects by a hop, relay across one of its two internal matched pairs, leave by a hop, and later do the same through the other pair.

The correction term 2β in Eq. (3) is essential: pricing hops by $\operatorname{dist} - \rho_u - \rho_v$ alone would give negative prices between defects inside a common blossom.

3.4 The search

By Theorem 3, relays are free and hops are non-negative. We now search over *walks* of the following kind, with a generic non-negative hop price c:

- **States** are L and the defects. Standing at a defect x means that we arrived at x by a relay from $\operatorname{mate}(x)$ and must leave x by a hop.
- **Start** at L with cost 0, and at every defect u with $\operatorname{mate}(u) = L$ with cost 0 (a free relay $L \Rightarrow u$).

- **Hop to a defect** $u \notin \{x, \mathrm{mate}(x)\}$, paying $c(x,u)$, then relay to $m = \mathrm{mate}(u)$: if m is a defect, the new state is m; if $m = R$ the walk is finished; if $m = L$ the move is discarded.
- **Hop to** R, paying $c(x,R)$, finishes the walk. From $x = L$ this costs d_{LR}.

With exact reduced costs, $c = \sigma$, we write Δ_σ for the cost of the cheapest finished walk. The state graph is complete on $|D| + 2$ nodes with non-negative arc costs, so Δ_σ is computed by Dijkstra's algorithm [3]. On a complete graph the original array-based form, which scans all unsettled labels to find the next state, runs in $O(|D|^2)$ time, and a priority queue [7] cannot improve on it, because every one of the $\Theta(|D|^2)$ arcs is relaxed. The matrix is the $|D| \times |D|$ matrix of reduced costs with its rows permuted by the matching.

Corollary 4. $\Delta_\sigma \le \Delta$ *on every shot, with or without blossoms.*

Proof. Every alternating path is a finished walk whose cost, $\sum_{\text{hops}} \sigma(h)$, is at most its true cost by Theorem 3. The search also admits walks that reuse a matched pair, which can only lower the minimum. Apply Lemma 2. □

In the example of Fig. 4, the search starts at L, hops to A (price 3), relays to B, hops to C (price 2.5), relays to D, hops to G (price 0.5), relays to E, hops to F (price 0) and relays to R, which finishes the walk. Hence $\Delta_\sigma = 6 = \Delta$.

4 Pricing hops without visiting the volume

Computing $\sigma(u,v)$ exactly requires $\mathrm{dist}(u,v)$ for all defect pairs, including far-apart pairs that only a sweep through the empty volume would connect. We replace dist by a certified lower bound $\widehat{\mathrm{dist}} \le \mathrm{dist}$.

Landmark potentials. Choose a small set of special vertices, or sets of vertices, of the detector graph, called *landmarks*. For a landmark ℓ, precompute the *potential* $h_\ell(x) = \mathrm{dist}(\ell, x)$ of every detector x, its distance to the nearest vertex of ℓ. The two boundary sides are always landmarks, with potentials $h_L(x) = \mathrm{dist}(x, L)$ and $h_R(x) = \mathrm{dist}(x, R)$, and we write K for the number of further landmarks. Every potential is 1-Lipschitz, $|h_\ell(u) - h_\ell(v)| \le \mathrm{dist}(u,v)$, by the triangle inequality, so

$$\widehat{\mathrm{dist}}(u,v) = \mathrm{LB}(u,v) = \max_{\ell} \big|h_\ell(u) - h_\ell(v)\big| \;\le\; \mathrm{dist}(u,v), \tag{7}$$

where the maximum runs over the $K+2$ landmarks. This is the lower bound of the ALT algorithm of Goldberg and Harrelson [11]. The potentials are computed once per detector error model by one multi-source Dijkstra search per landmark. They cost $K+2$ numbers per detector in memory and $O(K)$ time per pair at run time. Nothing about the geometry of the detector graph is assumed, so the bound remains valid on irregular graphs. Since L and R are landmarks, the boundary prices $\mathrm{dist}(u,L) = h_L(u)$ and $\mathrm{dist}(u,R) = h_R(u)$ are exact.

The priced search. We price each hop between defects by

$$\hat\sigma(u,v) = \max\big(\widehat{\mathrm{dist}}(u,v) - \rho_u - \rho_v + 2\beta(u,v),\, 0\big), \tag{8}$$

and run the search of Sec. 3.4 with $c = \hat\sigma$. We denote the result $\hat\Delta_\sigma$: the hat marks the same search as Δ_σ, with distances replaced by their landmark lower bounds. The clamp

at 0 is needed because $\widehat{\mathrm{dist}}$ may be much smaller than dist, so the unclamped expression can be negative and Dijkstra's algorithm requires non-negative arc costs. The clamp is safe because $\sigma \geq 0$, so $\hat{\sigma} \leq \sigma$ still holds.

Theorem 5. $\hat{\Delta}_\sigma \leq \Delta_\sigma \leq \Delta$ *on every shot.*

Proof. $\widehat{\mathrm{dist}} \leq \mathrm{dist}$ pointwise, so $\hat{\sigma} \leq \sigma$ on every arc, and shortest-path values are monotone in the arc costs. Combine with Corollary 4. □

Where the bound loses. The shortfall splits into two parts, $\Delta - \hat{\Delta}_\sigma = (\Delta - \Delta_\sigma) + (\Delta_\sigma - \hat{\Delta}_\sigma)$. The first is the blossom term of Eq. (5) together with the extra non-simple walks. The second is at most $\sum_h \left(\mathrm{dist}(h) - \widehat{\mathrm{dist}}(h)\right)$ over the hops h of the walk returned by the priced search. The search minimises over lower bounds, so it is biased towards the hops whose distances are most underestimated, and the quality of the bound is governed by the worst-certified directions of the landmark set. On a lattice with an L_1-like metric, the face potentials alone degenerate to an L_∞-type bound in which transverse motion is free; corner landmarks repair this at a few bytes per detector.

Early termination. Post-selection needs only to decide whether $\Delta \geq \tau$. The Dijkstra can then stop as soon as the smallest unsettled label is at least τ, and report $\hat{\Delta}_\sigma \geq \tau$.

5 Distance preservation under yoking

Post-selection can only benefit from a lower bound, since it never keeps a shot that the exact gap would discard. Code concatenation uses the gap differently, as a weight rather than as a filter. In a yoked surface code [10], n surface-code patches are concatenated with a simple outer code whose checks, the *yokes*, are products of the patches' logical operators measured by lattice surgery. In 1D, the yokes are the products of all logical X and of all logical Z operators of a row of patches, which forms an $[[n, n-2, 2]]$ outer code. In 2D the patches form an $n \times n$ grid with a yoke on every row and every column, which forms an $[[n^2, n^2 - 4n + 2, 4]]$ code. Each patch is decoded on its own, and a patch whose inner decode produced a logical error is called a *culprit* (or guilty); the others are *innocent*. A violated yoke reveals that the patches it covers contain an odd number of culprits, and the outer decoder chooses the set of patches to flip that explains all yokes at minimum total gap, i.e. the patches whose inner decodes were least certain. Every shot is used, so an underestimated gap can make an innocent patch look guilty to the outer decoder. In this section we show that this cannot reduce the distance exponent of a matchable outer code: the lower bound satisfies the same innocent-side inequality as the exact gap.

5.1 What the outer decoder needs

Consider one patch with true error E of weight $W(E)$, and write $\mathcal{D} = d_{LR}$ for the weight of the lightest logical operator. Let W^E be the minimum weight of a correction in the logical class of E (the correct class), and $W^{\bar{E}}$ the minimum weight in the other class. The decoder outputs the lighter of the two, so $W^\star = \min(W^E, W^{\bar{E}})$, $W^c = \max(W^E, W^{\bar{E}})$, and $\Delta = W^c - W^\star \geq 0$. Two facts hold on every shot:

(F1) $W^E \leq W(E)$, since E is itself a correction in its own class;

(F2) $W^\star + W^c \geq \mathcal{D}$, since by Lemma 2 the two matchings differ by an alternating L–R path, whose edges have total weight at most $W^\star + W^c$ and which contains an L–R chain.

If the patch's decode fails (the patch is a culprit), the decoder chose the wrong class, so $W^\star = W^{\bar{E}}$ and $W^c = W^E$. Then

$$\Delta = W^c - W^\star = 2W^c - (W^\star + W^c) \overset{\text{(F2)}}{\leq} 2W^c - \mathcal{D} = 2W^E - \mathcal{D} \overset{\text{(F1)}}{\leq} 2W(E) - \mathcal{D}.$$

If the decode succeeds (the patch is innocent), $W^\star = W^E$ and $W^c = W^{\bar{E}}$. Then

$$\Delta = W^c - W^\star = (W^\star + W^c) - 2W^\star \overset{\text{(F2)}}{\geq} \mathcal{D} - 2W^\star = \mathcal{D} - 2W^E \overset{\text{(F1)}}{\geq} \mathcal{D} - 2W(E).$$

An estimate $\hat{g}$ handed to the outer decoder in place of Δ preserves both inequalities if

$$\begin{aligned} &\text{(C)} \quad \hat{g} \leq \Delta && \text{(culprit side)},\\ &\text{(I)} \quad \hat{g} \geq \mathcal{D} - 2W^\star && \text{(innocent side)}. \end{aligned}$$

Proposition 6. *Consider an outer code with noiseless checks and distance D_o, decoded by minimum-weight matching in which patch i has weight $\hat{g}_i$, where every $\hat{g}_i$ satisfies (C) and (I). If the outer decoder fails, then the inner errors satisfy $\sum_i W(E_i) \geq \frac{1}{2}\mathcal{D}\, D_o$.*

Proof. Let F be the set of culprit patches, and let C be the set of patches flipped by the outer decoder. Both are consistent with the outer syndrome, so minimality gives $\sum_{i\in C} \hat{g}_i \leq \sum_{i\in F} \hat{g}_i$, i.e. $\sum_{i\in C\setminus F} \hat{g}_i \leq \sum_{i\in F\setminus C} \hat{g}_i$. Patches in $C \setminus F$ are innocent and those in $F \setminus C$ are culprits, so by (I), (F1) and (C),

$$\sum_{i\in C\setminus F} (\mathcal{D} - 2W(E_i)) \;\leq\; \sum_{i\in F\setminus C} (2W(E_i) - \mathcal{D}), \qquad \text{i.e.} \qquad \sum_{i\in C\oplus F} W(E_i) \;\geq\; \tfrac{1}{2}\mathcal{D}\,|C \oplus F|.$$

A failure means that $C \oplus F$ is a nontrivial outer logical, so $|C \oplus F| \geq D_o$. □

A single patch fails at inner fault weight $\mathcal{D}/2$. With the $[[n, n-2, 2]]$ (1D) and $[[n^2, n^2-4n+2, 4]]$ (2D) yokes, a failure needs inner fault weight $\mathcal{D}$ and $2\mathcal{D}$ respectively: the doubling and quadrupling of the exponent observed with the exact gap [10]. Any lower bound satisfies (C) automatically. The rest of this section proves (I) for $\hat{\Delta}_\sigma$. The yoked-code model of Ref. [10] also includes noisy yoke measurements, which we do not treat here.

5.2 The cluster lemma

The proof of (I) uses one property of the dual state beyond feasibility, tightness and complementary slackness. Each blossom S with $y_S > 0$ has *children*, which are its maximal sub-blossoms and its defects that lie in no sub-blossom. Its children form an odd cycle $S^1, \ldots, S^k$, and consecutive children are joined by defects $u \in S^i$, $v \in S^{i+1}$ with $\sigma(u, v) = 0$. Edmonds' algorithm creates blossoms from such tight odd cycles, and sparse blossom grows or shrinks a blossom's interior as a unit, so the cycle stays tight while $y_S > 0$ [5, 12]. For a pair of defects let

$$g(u, v) = \mathrm{dist}(u, v) - \rho_u - \rho_v, \qquad \kappa(u, v) = \rho_u + \rho_v - 2\beta(u, v) = \sum_{X \text{ crossed by } (u,v)} y_X,$$

where X ranges over the dual sets: the singletons $\{v\}$, with value y_v, and the blossoms. Both have a simple picture (Fig. 5a). Draw around every defect v a ball of its total radius ρ_v, the extent of the region that the decoder grew around v. Then $g(u, v)$ is the length of the empty stretch between the balls of u and v: positive if the two regions never met, and zero or negative if they touch or overlap. The quantity $\kappa(u, v)$ is the dual charged to

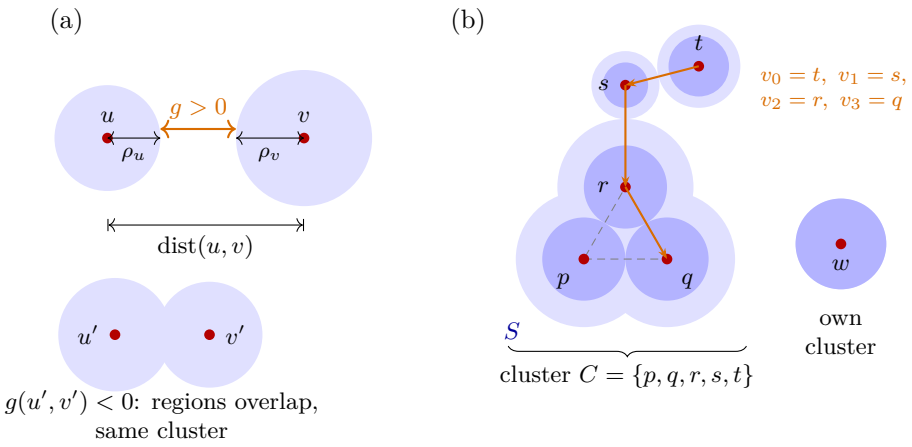


Figure 5: (a) The quantity $g(u, v) = \mathrm{dist}(u, v) - \rho_u - \rho_v$ is the length of the empty stretch between the total-radius balls of two defects; it is non-positive exactly when the decoder's regions around them touch or overlap. (b) A cluster: the blossom $S = \{p, q, r\}$ (dark balls of radius y_v inside a shell of thickness y_S; its odd cycle of tight edges is dashed), and the defects s and t, whose regions touch S and each other. The defect w touches nothing and is its own cluster. The walk used in the proof of Lemma 7, from t to q, steps between touching top-level objects ($t \to s \to r$) and then along the cycle of S ($r \to q$). It enters every dual set at most once and never returns to it.

the pair, as in Sec. 2.3: in the forest of Fig. 3, it is the sum of the values on the tree path from u to v below their lowest common ancestor. The two differ only by the shared mass, $g = \mathrm{dist} - \kappa - 2\beta$. Feasibility reads $\mathrm{dist} \geq \kappa$, and the cycle edges of a blossom satisfy $\mathrm{dist} = \kappa$.

A *cluster* is a connected component of the graph on D whose edges are the pairs with $g \leq 0$: a maximal group of defects whose regions touch, directly or through other defects of the group (Fig. 5b). A blossom's cycle edges have $g = -2\beta \leq 0$, so by induction every blossom lies inside one cluster. In the forest of Fig. 3, a cluster is therefore a union of whole trees, and its *dual mass*

$$Y(C) = \sum_{v \in C} y_v + \sum_{S \subseteq C} y_S$$

is the sum of all node values in those trees. Strong duality then gives $\sum_C Y(C) = W^\star$.

Lemma 7 (Cluster lemma)**.** *For any two defects a, b in the same cluster C,* $\mathrm{dist}(b, a) + \rho_a + \rho_b \leq 2\,Y(C)$.

Proof. We build a sequence $b = v_0, v_1, \ldots, v_m = a$ of defects in C, a walk through the cluster as in Fig. 5b, with two properties:

(i) every step satisfies $\mathrm{dist}(v_i, v_{i+1}) \leq \kappa(v_i, v_{i+1})$, i.e. the step is no longer than the dual charged to it;

(ii) for every dual set X the indices i with $v_i \in X$ form a contiguous interval: once the walk leaves X it never comes back.

Property (i) is not a general fact, since feasibility gives the opposite inequality $\mathrm{dist} \geq \kappa$. It holds for the two kinds of step that the construction below uses: steps between touching top-level objects, for which it follows from $g \leq 0$ and $\beta = 0$, and steps along a blossom's odd cycle, which are tight, $\mathrm{dist} = \kappa$. Call the maximal blossoms of C and its defects

outside every blossom the *top-level objects* of C. Two defects in different top-level objects share no blossom, so $\beta = 0$ between them, and $\kappa = \rho_u + \rho_v$. An edge between them with $g \le 0$ therefore satisfies dist $\le \rho_u + \rho_v = \kappa$. Since C is connected, the top-level objects form a connected graph under such edges. Pick a simple path of objects $T_0 \ni b, \dots, T_r \ni a$, with connecting edges (q_i, p_{i+1}), $q_i \in T_i$, $p_{i+1} \in T_{i+1}$, and set $p_0 = b$, $q_r = a$. Inside each T_i, route from p_i to q_i as follows. In a singleton, $p = q$ and we emit it. In a blossom, let p lie in child S^s and q in child S^t. Walk around the odd cycle from S^s to S^t in one direction, crossing each cycle edge (for which dist $= \kappa$), and route recursively inside each child on the way, from the vertex where we entered it to the vertex where we leave it. Every object and every child is visited in a single stretch, and blossoms are laminar, so every dual set is visited contiguously.

By the triangle inequality, $\mathrm{dist}(b, a) \le \sum_i \kappa(v_i, v_{i+1}) = \sum_X y_X\, c_X$, where c_X counts the steps that cross X. Also $\rho_b = \sum_{X \ni b} y_X$ and $\rho_a = \sum_{X \ni a} y_X$. For each X, the quantity $[b \in X] + c_X + [a \in X]$ counts the entry into and the exit from the contiguous run of the sequence inside X. It is therefore 0 or 2, and it is nonzero only if X meets C, in which case $X \subseteq C$. Summing over X gives the claim. □

The cycle structure is essential. For arbitrary LPs and optimal duals that lack it, the inequality can fail.

5.3 The innocent side for the lower bound

Proposition 8. *Suppose that L is a landmark and that any saturation cap on the potentials is at least d_{LR}. Then on every shot, with or without blossoms,*

$$\hat{\Delta}_\sigma \;\ge\; d_{LR} - 2W^\star.$$

Proof. Step 1: relax the search to a path problem. Let $\hat{g}(u, v) = \widehat{\mathrm{dist}}(u, v) - \rho_u - \rho_v$ for defects and $\hat{g}(u, X) = h_X(u) - \rho_u$ for $X \in \{L, R\}$. Since $\beta \ge 0$, every hop price satisfies $\hat{\sigma} = \max(\hat{g} + 2\beta, 0) \ge \max(\hat{g}, 0)$. Relays have $\hat{g} \le g = -2\beta \le 0$ between defects and $\hat{g} = 0$ to the boundary, by tightness. Let $\mathcal{G}$ be the complete graph on $D \cup \{L, R\}$ with edge weights $w = \max(\hat{g}, 0)$ and $w(L, R) = d_{LR}$. The clamp at 0 is essential here. The value $\hat{g}$ is negative on every relay between defects with $\beta > 0$, and can be negative on hops between defects whose regions overlap. With negative weights, a shortest walk could gain by cycling through such edges, and Step 2 below could not discard the edges of the path that it does not use. With the clamp, all weights are non-negative and relays are free. A finished walk of the search, with its relays inserted, is an L–R walk in $\mathcal{G}$ that costs no more than the walk, even if it reuses a matched pair. Hence $\hat{\Delta}_\sigma \ge \Phi$, the length of a shortest L–R path in $\mathcal{G}$. (Φ is only a proof device. It is never computed in *fastgap*. Φ is exactly the output computed in [16].)

Step 2: cut the path at clusters. If the shortest path is the edge (L, R), then $\Phi = d_{LR}$ and we are done. Otherwise write it as $L, x_1, \dots, x_n, R$. Let $b_1 = x_1$, let C_1 be its cluster, and let a_1 be the *last* vertex of the path in C_1. Let b_2 be the vertex after a_1, C_2 its cluster, a_2 the last vertex in C_2, and so on up to $a_k = x_n$. Since the weights are non-negative we may take the shortest path to be simple, so it visits every defect at most once. The clusters $C_1, \dots, C_k$ are distinct: a_j is the last vertex of the path in C_j, so no later vertex, in particular no $b_{j'}$ with $j' > j$, lies in C_j. The edges (L, b_1), (a_j, b_{j+1}) and (a_k, R) are edges of the path and all weights are non-negative, so their total weight is at most Φ.

Step 3: telescope the L potential. Let $\bar{h} = \min(h_L, d_{LR})$, which is 1-Lipschitz. Saturation at a cap $\ge d_{LR}$ commutes with this truncation, so $\widehat{\mathrm{dist}}(u, v) \ge |\bar{h}(u) - \bar{h}(v)|$ for every

pair of defects. Hence

$$\begin{aligned} w(L, b_1) &\geq \bar{h}(b_1) - \rho_{b_1}, \\ w(a_j, b_{j+1}) &\geq \bar{h}(b_{j+1}) - \bar{h}(a_j) - \rho_{a_j} - \rho_{b_{j+1}}, \\ w(a_k, R) &\geq h_R(a_k) - \rho_{a_k} \geq d_{LR} - \bar{h}(a_k) - \rho_{a_k}, \end{aligned}$$

where the last step uses $h_L + h_R \geq d_{LR}$. Inside each cluster, 1-Lipschitz continuity and Lemma 7 give $\bar{h}(a_j) - \bar{h}(b_j) \leq \mathrm{dist}(b_j, a_j) \leq 2Y(C_j) - \rho_{a_j} - \rho_{b_j}$. Now write d_{LR} as the telescoping sum $\bar{h}(b_1) + \sum_j \left(\bar{h}(a_j) - \bar{h}(b_j)\right) + \sum_{j<k} \left(\bar{h}(b_{j+1}) - \bar{h}(a_j)\right) + \left(d_{LR} - \bar{h}(a_k)\right)$ and bound each term as above. Every ρ appears once with each sign and cancels, so

$$d_{LR} \;\leq\; \Phi + 2\sum_j Y(C_j) \;\leq\; \Phi + 2W^\star \;\leq\; \hat{\Delta}_\sigma + 2W^\star,$$

because the clusters C_j are distinct, so each dual mass $Y(C_j)$ is counted once and their sum is at most $\sum_C Y(C) = W^\star$. □

Combined with Theorem 5 and Proposition 6, this shows that using $\hat{\Delta}_\sigma$ in place of Δ preserves the distance exponent of 1D and 2D yoked surface codes. Only the prefactor of the logical error rate can change. The proof uses no landmark other than L. The other landmarks only improve tightness, i.e. the prefactor. Three conditions matter in practice: the boundary potentials must not be saturated below d_{LR}, distances must never be overestimated (which rules out reweighting the graph, as correlated matching does, without rebuilding the index), and the outer decoder must return a minimum-weight correction. The last condition holds for the matching-based outer decoders of 1D and 2D yokes, but not for belief propagation.

6 Implementation

We implemented the method as *fastgap*, a fork of PyMatching v2 [12] with a C++ core and Python bindings. The upstream decoder is unchanged apart from exposing two internal functions; all upstream tests pass, and fastgap's predictions and weights are bit-identical to upstream PyMatching on the same detector error model.

6.1 Index (once per detector error model)

The decoder, the index and the exact two-decode baseline are all built from the same detector error model, so they see identical edges, weights and integer units. The index reads the distance graph from the decoder's own internal graph and stores the boundary potentials h_L, h_R and the K landmark potentials, node-major, in `int64`, with an `int32` copy saturated at $2^{31} - 1$ for pricing. Saturation preserves the bound because $|\min(a, C) - \min(b, C)| \leq |a - b|$. Landmark placement is pluggable. The built-in strategies are `boundary_only` (just L and R), `faces` (time faces and the spatial faces not covered by L, R), `faces_corners` (faces plus the eight spacetime corners; the default for memory experiments), `fps` (farthest-point sampling in the graph metric, needing no coordinates), and `greedy_cover` (greedy maximisation of the bound over hop pairs sampled from real shots, in the spirit of the "maxcover" landmark selection of Ref. [11]). Correctness never depends on the choice.

6.2 Dual state extraction

After sparse blossom has run to completion, and *before* it shatters its blossoms to extract the matched edges (which destroys the region tree), we walk the chain of regions from each defect's leaf up to its outermost blossom. We read each region's radius, assign dense identifiers to blossoms, and store per defect its y_v, its ancestor blossoms top-down with prefix sums, and ρ_v. The shared mass $\beta(u, v)$ is then a walk over the common prefix of two ancestor lists, and returns 0 immediately when the outermost blossoms differ, which is the common case. $W^\star = \sum y$ is read in the same pass. We then let the upstream code shatter the blossoms and build mate$(\cdot)$, and the boundary side of each boundary match, from the matched edges' observable masks.

6.3 The search

Algorithm 1 is a dense Dijkstra over the $|D| + 2$ states, with rows priced lazily as states are settled. Several implementation details matter for speed without changing any price:

- **Labels per hop target.** Every hop target u leads, by its relay, to exactly one place, mate(u) or R, so the label of state mate(u) can be stored at u's position. Relaxing a row is then one contiguous, branch-free min pass, into which the arg min for the next iteration is fused.
- **Row pricing in one pass.** The potentials of all hop targets are gathered once per shot into a potential-major block, so the landmark bound for a row is $K + 2$ contiguous vectorised passes.
- **Deterministic ties.** Ties between finished walks are broken by (iteration, defect index), so the returned walk never depends on the number of threads.

Row pricing dominates the cost, $O(|D|\,K)$ per row, so the search costs $O(|D|^2 K)$ in the worst case, but thread 0 never needs the rows of states whose label exceeds the current best. The work is independent of the number of detectors $|V|$.

6.4 Intra-shot parallelism

Real-time applications need low latency per shot rather than throughput, so T threads cooperate on *each* shot. Thread 0 runs the Dijkstra (Search). The other $T - 1$ threads prefetch price rows (Prefetch): each repeatedly claims, by compare-and-swap, the unsettled state with the smallest published label, which is the state thread 0 is likely to settle next, and prices its row into a shared cache. When thread 0 settles a state, it uses the cached row if it is ready, prices the row itself if no thread has claimed it, or waits for the claimant (GetRow). Prefetching is speculative: a prefetched row may never be used if the search ends first. Rows do not depend on who computed them, so all thread counts give bit-identical results. Shared rows are stored as `int32` saturated at $d_{LR} + 1$. This is exact rather than approximate: once L is settled the best finished walk costs at most d_{LR}, so no label at or above $d_{LR} + 1$ is ever settled or returned. Idle threads spin for a short period before sleeping, since a condition-variable wake-up costs several microseconds, more than a whole search on small shots; shots with fewer than 16 defects are searched by one thread. We also tried a design in which each thread owns a slice of the targets and all threads perform an all-reduce per settled state; on our machine the all-reduce alone cost about half of one row's work, limiting the speed-up at $d = 17$ to 1.3×.

Algorithm 1 Gap lower bound from the decoder's final state with T cooperating threads. Thread 0 runs SEARCH; threads $1, \dots, T-1$ run PREFETCH. Row pricing (lines 23 and 31) is the only parallelised work: different rows are priced concurrently by different threads. The relaxation loop (line 9) is sequential and vectorised.

Require: defects D; mate, ρ, β from the decode; potentials $h_L, h_R, h_1, \dots, h_K$ and d_{LR} from the index; optional threshold τ
Shared: labels λ published by thread 0; row cache with a state per row in $\{\text{FREE}, \text{CLAIMED}, \text{READY}\}$

1: **procedure** SEARCH ▷ thread 0
2: best $\leftarrow d_{LR}$ ▷ bare L–R chain
3: $\lambda(L) \leftarrow 0$; $\lambda(u) \leftarrow 0$ for all u with $\text{mate}(u) = L$; $\lambda(x) \leftarrow \infty$ otherwise
4: **while** some state is unsettled and $\min \lambda <$ best **do**
5: settle $x \leftarrow \arg\min \lambda$ over unsettled states
6: **if** τ given and $\lambda(x) \geq \tau$ **then return** τ (censored)
7: **end if**
8: row $\leftarrow$ GETROW(x)
9: **for** each defect $u \notin \{x, \text{mate}(x)\}$ with $\text{mate}(u) \neq L$ **do**
10: **if** $\text{mate}(u) = R$ **then** best $\leftarrow \min(\text{best}, \lambda(x) + \text{row}[u])$
11: **else** $\lambda(\text{mate}(u)) \leftarrow \min\left(\lambda(\text{mate}(u)), \lambda(x) + \text{row}[u]\right)$
12: **end if**
13: **end for**
14: best $\leftarrow \min(\text{best}, \lambda(x) + h_R(x) - \rho_x)$ ▷ hop to R
15: publish the labels that decreased
16: **end while**
17: signal PREFETCH threads to stop; **return** $\hat{\Delta}_\sigma =$ best
18: **end procedure**
19: **procedure** PREFETCH ▷ threads $1, \dots, T-1$
20: **while** the search is running **do**
21: $x \leftarrow$ the unsettled FREE state with the smallest published label
22: **if** compare-and-swap of x from FREE to CLAIMED succeeds **then**
23: price row x into the cache; mark it READY
24: **end if**
25: **end while**
26: **end procedure**
27: **function** GETROW(x)
28: **if** row x is READY **then return** it
29: **end if**
30: **if** compare-and-swap of x from FREE to CLAIMED succeeds **then**
31: price row x; **return** it
32: **end if**
33: wait until row x is READY; **return** it ▷ claimed by a prefetcher
34: **end function**

Price row x: for every hop target u, $\text{row}[u] = \max\left(\max_\ell |h_\ell(x) - h_\ell(u)| - \rho_x - \rho_u + 2\beta(x,u),\ 0\right)$, the max over L, R and the K landmarks; for $x = L$, $\text{row}[u] = h_L(u) - \rho_u$.

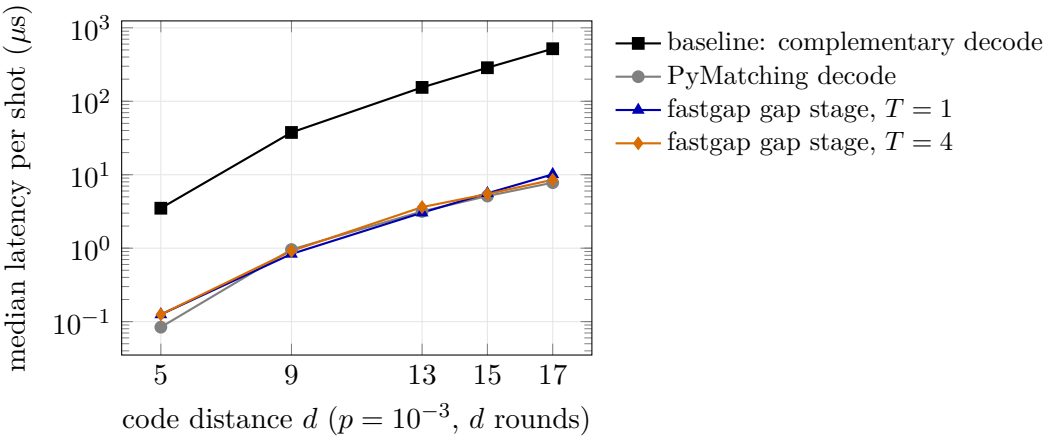


Figure 6: Median per-shot latency of the gap computation for rotated surface-code memory at $p = 10^{-3}$. The baseline's cost of the gap is its decode in the complementary class; fastgap's is its gap stage, which runs after an ordinary PyMatching decode that both methods share (gray).

7 Numerical results

Setup. We use Stim-generated [8] rotated surface-code memory experiments (Stim task `rotated_memory_x`) with d rounds and uniform circuit-level depolarising noise of strength p, decomposed into graphlike detector error models. The exact baseline is the method used in the cultivation and yoking codebases [9, 10]: two PyMatching decodes of the same detector error model with the observable turned into an extra detector, switched on and off. The decode in the class of $M^\star$ reproduces the ordinary decode; the decode in the complementary class is the cost the baseline pays for the gap. The index uses the `faces_corners` strategy ($K = 12$ landmarks plus L, R). All timings are compute-only per-shot stage timers (`steady_clock`) on an Apple M5 Pro (macOS, Apple clang 21), after 1000 warm-up shots, with shots decoded one after another; T counts all threads cooperating on a shot. Each configuration uses 2×10^4 shots.

Latency. Figure 6 and Table 1 compare the latency of the gap computation with the complementary decode of the baseline. fastgap's gap stage is 11–51× faster at the median with one thread, and 61× with four threads at $d = 17$. The complementary decode grows with the spacetime volume, whereas the search touches only the defects, so the advantage widens with distance up to $d = 13$ and then stays at about 51× for $T = 1$. At $d = 17$, $p = 10^{-3}$ (88 defects on average) the complete pipeline, decode plus gap bound, takes 18.0 μs (median) with one thread and 16.8 μs with four, against 528 μs for the exact two-decode baseline. At $d = 15$ (60 defects) it takes 10.8 μs, against 292 μs. Measured end to end, with syndromes sent to a separate server process through a pipe and verdicts returned, the median round trip at $d = 17$ is 34 μs ($T = 1$) and 30 μs ($T = 4$), of which roughly 12 μs is fixed I/O and Python overhead.

Four threads help only on the largest shots: at $d = 17$ the gap stage drops from 10.1 to 8.5 μs. Since a row costs only $K + 2$ vectorised passes over the targets, reading a row that another core priced costs nearly as much as pricing it locally, and at $d = 7$ and $d = 13$ four threads are slower than one (2.4 vs. 1.7 μs and 3.6 vs. 3.0 μs). At $d \leq 9$ most shots fall below the parallel grain and are searched by one thread. Scaling beyond four or five threads is limited on our machine by the core cluster: prefetched rows that cross to a cluster with a different L2 cache cost thread 0 more to read than to compute.

configuration	defects	PyMatching decode	baseline gap	fastgap $T=1$	fastgap $T=4$	speed-up $T=1$	speed-up $T=4$
$d=5$, $p=5\times10^{-3}$	8.4	0.83 / 5.0	6.92 / 14.0	0.62 / 1.7	0.62 / 2.3	11.1 / 8.2	11.1 / 6.0
$d=7$, $p=5\times10^{-3}$	25.3	3.21 / 12.8	23.92 / 42.9	1.67 / 4.0	2.42 / 4.9	14.3 / 10.7	9.9 / 8.8
$d=5$, $p=10^{-3}$	1.8	0.08 / 1.2	3.50 / 9.5	0.12 / 0.8	0.12 / 0.7	28.0 / 12.6	28.0 / 13.3
$d=9$, $p=10^{-3}$	12.0	0.96 / 3.8	37.62 / 60.7	0.83 / 2.2	0.92 / 3.0	45.1 / 27.0	41.1 / 20.0
$d=13$, $p=10^{-3}$	38.3	3.17 / 7.8	154.46 / 226.9	3.04 / 6.7	3.62 / 6.8	50.8 / 33.8	42.6 / 33.6
$d=15$, $p=10^{-3}$	59.6	5.12 / 10.8	286.17 / 421.0	5.58 / 11.5	5.46 / 10.2	51.3 / 36.5	52.4 / 41.2
$d=17$, $p=10^{-3}$	87.8	7.79 / 14.4	518.92 / 787.9	10.12 / 19.6	8.54 / 15.5	51.3 / 40.1	60.7 / 51.0

Table 1: Per-shot latency in μs (median / p99) over 2×10^4 shots: the ordinary PyMatching (minimum-weight matching) decode that both methods run, the baseline's complementary-class decode, and fastgap's gap stage; and the speed-up of the gap computation (baseline complementary-decode latency divided by fastgap gap-stage latency, median / p99). "Defects" is the mean number per shot.

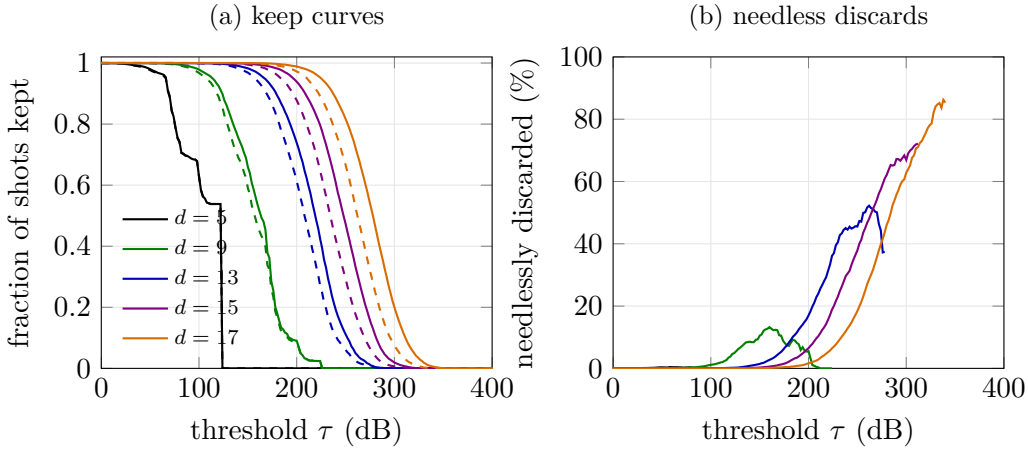


Figure 7: Post-selection at $p = 10^{-3}$, 2×10^4 shots per distance. (a) Fraction of shots whose gap is at least τ, for the exact gap (solid) and the fastgap lower bound (dashed); the gaps of a memory experiment grow with d. (b) Shots that the exact gap keeps but the bound discards ($\hat{\Delta}_\sigma < \tau \leq \Delta$), as a percentage of the shots that the exact gap keeps. Curves end where the exact gap keeps fewer than 0.5% of the shots.

Post-selection. Figure 7 translates the bound into the quantity that matters for post-selection, the fraction of shots kept at a threshold τ. Because $\hat{\Delta}_\sigma \leq \Delta$, the shots kept by the bound are always a subset of those kept by the exact gap, so thresholding on the bound never admits a larger number of logical errors. The cost is a shift of the keep curve to larger thresholds, which is negligible at $d = 5$ and grows with the distance. Figure 7b shows it as the fraction of the shots kept by the exact gap that the bound discards needlessly. At the threshold at which the exact gap keeps 99% of the shots, this fraction is at most 1.3% for $d \leq 17$. At the threshold at which the exact gap keeps 90%, it is 0.3%, 4.9%, 7.9%, 9.9% and 11.7% for $d = 5, 9, 13, 15, 17$.

8 Discussion

Using a lower bound in place of the gap. For post-selection, a lower bound is a conservative substitute: at any threshold it keeps a subset of the shots the exact gap would keep, and in particular never more logical errors. The relevant cost is the extra retry rate at equal logical error per kept state, which requires recalibrating the threshold for the

bound. Figure 7b shows that at large distance this cost is not negligible with the default twelve landmarks, so landmark placement is the main lever for post-selection. For code concatenation, where the gap is used as a weight for the outer decoder, an underestimated gap on an innocent patch can shift blame onto it. Section 5 settles the asymptotic question for matchable outer codes: the distance exponent is preserved. The effect on the prefactors of yoked-code logical error rates, and on the pseudothreshold of hierarchical codes with soft-information outer decoding [16], must be measured. In all cases each estimator needs its own calibration from gap to failure probability.

Relation to cluster-based soft outputs. The cluster gap of Meister *et al.* [16] and its refinement by Kishi *et al.* [15] derive soft information from the clusters of a fast decoder, and are typically cheaper than a second matching. Our bound instead targets the complementary gap of the MWPM decoder itself, the quantity used by the published cultivation and yoking pipelines, and comes with a certificate. The search operates on the dual state of any blossom-type matching decoder and adds only a few microseconds of CPU time after it.

Limitations and future work. fastgap currently supports uncorrelated matching, non-negative weights and codes whose observable cannot be flipped by a closed loop that avoids the boundary. All results here are for rotated surface-code memory; the escape-stage detector error models of magic state cultivation [9, 13], which contain a colour-code region grafted into a surface code, are supported by the index construction but have not yet been benchmarked, and landmark placement on such irregular graphs will need care. The latency results were measured on a single desktop-class CPU without thread pinning. Planned experiments apply the bound to the three applications above: the relative retry rate of cultivation at equal logical error rate, the fitted logical error rates of 1D and 2D yoked surface codes, and the pseudothreshold of a surface code concatenated with a quasi-cyclic lifted-product code [16].

9 Conclusion

The complementary gap of a matching decoder can be bounded from below, rigorously and on every shot, using only the final state of a single ordinary decode. The decoder's own dual variables turn the gap into a non-negative shortest-path problem on the defects, landmark potentials price that problem without visiting the empty spacetime volume, and a dense, cooperatively parallel Dijkstra solves it in about a tenth of a microsecond to ten microseconds for distances up to 17 at $p = 10^{-3}$, between one and two orders of magnitude faster than the second decode it replaces. The bound keeps the distance exponent of yoked surface codes, and its looseness depends only on the choice of landmarks.

Code availability

The fastgap source code, benchmark scripts and raw per-shot data are available at https://github.com/sumeetshirgure/fastgap.

References

[1] Héctor Bombín, Mihir Pant, Sam Roberts, and Karthik I. Seetharam. Fault-tolerant postselection for low-overhead magic state preparation. *PRX Quantum*, 5:010302, Jan 2024. DOI: 10.1103/PRXQuantum.5.010302. URL https://link.aps.org/doi/10.1103/PRXQuantum.5.010302.

[2] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43(9):4452–4505, 2002. DOI: 10.1063/1.1499754.

[3] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959. DOI: 10.1007/BF01386390.

[4] Jack Edmonds. Maximum matching and a polyhedron with 0,1-vertices. *Journal of Research of the National Bureau of Standards B*, 69:125–130, 1965.

[5] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17: 449–467, 1965. DOI: 10.4153/CJM-1965-045-4.

[6] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Phys. Rev. A*, 86:032324, 2012. DOI: 10.1103/PhysRevA.86.032324.

[7] Michael L. Fredman and Robert Endre Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987. DOI: 10.1145/28869.28874.

[8] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021. DOI: 10.22331/q-2021-07-06-497.

[9] Craig Gidney, Noah Shutty, and Cody Jones. Magic state cultivation: growing t states as cheap as cnot gates. *arXiv preprint arXiv:2409.17595*, 2024.

[10] Craig Gidney, Michael Newman, Peter Brooks, and Cody Jones. Yoked surface codes. *Nature Communications*, 16(1):4498, 2025.

[11] Andrew V. Goldberg and Chris Harrelson. Computing the shortest path: A* search meets graph theory. In *Proceedings of the Sixteenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 156–165, 2005.

[12] Oscar Higgott and Craig Gidney. Sparse blossom: correcting a million errors per core second with minimum-weight matching. *Quantum*, 9:1600, 2025. DOI: 10.22331/q-2025-01-20-1600.

[13] Yutaka Hirano, Riki Toshio, Tomohiro Itogawa, and Keisuke Fujii. Efficient magic state cultivation with lattice surgery. *arXiv preprint arXiv:2510.24615*, 2025.

[14] Adrian Hutter, James R. Wootton, and Daniel Loss. Efficient markov chain monte carlo algorithm for the surface code. *Phys. Rev. A*, 89:022326, Feb 2014. DOI: 10.1103/PhysRevA.89.022326. URL https://link.aps.org/doi/10.1103/PhysRevA.89.022326.

[15] Kaito Kishi, Riki Toshio, Jun Fujisaki, Hirotaka Oshima, Shintaro Sato, and Keisuke Fujii. Even more efficient soft-output decoding with extra-cluster growth and early stopping, 2026. URL https://arxiv.org/abs/2602.03336.

[16] Nadine Meister, Christopher A Pattison, and John Preskill. Efficient soft-output decoders for the surface code. *arXiv preprint arXiv:2405.07433*, 2024.

[17] Christopher A. Pattison, Anirudh Krishna, and John Preskill. Hierarchical memories: Simulating quantum LDPC codes with local gates. *Quantum*, 9:1728, 2025. arXiv:2303.04798.

[18] David Poulin. Optimal and efficient decoding of concatenated quantum block codes.

Phys. Rev. A, 74:052333, Nov 2006. DOI: 10.1103/PhysRevA.74.052333. URL https://link.aps.org/doi/10.1103/PhysRevA.74.052333.